Basic (embeddings as geometry) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The exercise has two separate pieces: calling an embeddings API, and computing **cosine similarity** between two vectors.

Cosine similarity measures the angle between two vectors, not their length — this matters because it means a short sentence and a long sentence about the same topic can still score as "close," since only the direction matters, not the size. The formula is:

```
cosine_similarity(a, b) = dot(a, b) / (norm(a) * norm(b))
```

Where `dot(a, b)` is the sum of `a[i] * b[i]` over every position, and `norm(a)` is the length of vector `a` (square root of the sum of its squares).

The exact pieces:

- `client.embeddings.create(model="text-embedding-3-small", input=text)` — the vector is in `response.data[0].embedding`.
- `sum(x * y for x, y in zip(a, b))` for the dot product.
- `math.sqrt(sum(x * x for x in a))` for a vector's norm.
- `itertools.combinations(sentences, 2)` — every unique pair, with no risk of comparing a sentence to itself.

Think about what happens once this stops being a 5-sentence toy script. Calling `client.embeddings.create()` once per sentence, in a loop, is 5 separate network round trips for 5 short sentences — each one carries its own latency, and at real scale (hundreds or thousands of pieces of text) that adds up to a genuinely slow pipeline, one request at a time, for no reason.

The `input` parameter to `embeddings.create()` accepts a *list* of strings, not just one — so all 5 sentences can be embedded in a single API call, and the response comes back as a list of vectors in the same order. This is the same idea as Doc01's "fail at startup, not later," just applied to network calls: do the expensive thing once, in bulk, instead of many times for no benefit.

There's a second thing worth knowing, not just doing: OpenAI's embedding vectors already come back unit-length (`norm(a) == 1.0` for every one of them). That means `cosine_similarity(a, b)` mathematically simplifies to just `dot(a, b)` — the division by `norm(a) * norm(b)` divides by `1.0 * 1.0`, which changes nothing. A real pipeline that calls this same embedding model thousands of times a day skips the two `math.sqrt()` calls entirely and just uses the dot product, once this assumption has actually been checked and confirmed — not assumed blindly, since not every embedding model guarantees unit-length vectors.

The design question underneath both of these: **what's genuinely necessary work, versus work you're doing out of habit?** A general-purpose `cosine_similarity()` that handles any vector, from any model, is the safe default to write first. Once you know exactly which model you're using and that it always returns unit vectors, dropping the redundant normalization is a real, measurable optimization — not premature.

Hint 2 — The plan, and almost the whole thing

```
function get_embedding(text) -> list[float]:
    call the embeddings API, return response.data[0].embedding

function cosine_similarity(a, b) -> float:
    dot = sum of a[i] * b[i]
    norm_a = sqrt(sum of a[i]^2)
    norm_b = sqrt(sum of b[i]^2)
    return dot / (norm_a * norm_b)

main:
    sentences = [s1, s2, s3, s4, s5]   # s1, s2 are the similar pair
    embeddings = {s: get_embedding(s) for s in sentences}
    for each unique pair (s_a, s_b) from itertools.combinations(sentences, 2):
        score = cosine_similarity(embeddings[s_a], embeddings[s_b])
        print(s_a, s_b, score)
    confirm the (s1, s2) pair has the highest score of all pairs
```

```
# embedding_similarity_practice.py
import math
from itertools import combinations
from openai import OpenAI
```

```
client = OpenAI()

def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)
```

Write your 5 sentences and the `combinations`-based comparison loop yourself, then compare both against the Solution.

Once that's working, cut the unnecessary work:

```
sentences = [s1, s2, s3, s4, s5]   # s1, s2 are the similar pair

batch-embed every sentence in one API call:
    response = client.embeddings.create(model=..., input=sentences)
    embeddings = [item.embedding for item in response.data]   # same order

function fast_cosine_similarity(a, b) -> float:
    # a and b are already unit-length (this model guarantees it) --
    # skip both norm() calls entirely
    return sum of a[i] * b[i]

for each unique pair:
    score = fast_cosine_similarity(...)
    print the pair and score

sanity check once, don't just assume:
    assert abs(math.sqrt(sum(x * x for x in embeddings[0])) - 1.0) < 1e-6
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# embedding_similarity_practice.py
import math
from itertools import combinations
from openai import OpenAI

client = OpenAI()

SENTENCES: list[str] = [
    # your 5 sentences here
]

def get_embeddings_batch(texts: list[str]) -> list[list[float]]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=texts
    )
    embeddings = []
    for item in response.data:
        embeddings.append(item.embedding)
    return embeddings

def fast_cosine_similarity(a: list[float], b: list[float]) -> float:
    # your turn: these vectors are already unit-length (verified below) --
    # just return the dot product, no norm() calls needed
    ...

# one-time sanity check that the "already unit-length" assumption really holds
# for this model, instead of trusting it blindly
def assert_unit_length(vector: list[float]) -> None:
    squares = 0.0
    for x in vector:
        squares = squares + x * x
    norm = math.sqrt(squares)
    assert abs(norm - 1.0) < 1e-6, f"expected a unit vector, got norm={norm}"
```

Fill in `fast_cosine_similarity`, embed `SENTENCES` in one batch call, run `assert_unit_length` on the first vector, then score every pair and compare all of your finished versions against the Solution.

Full solution: Show me the solution